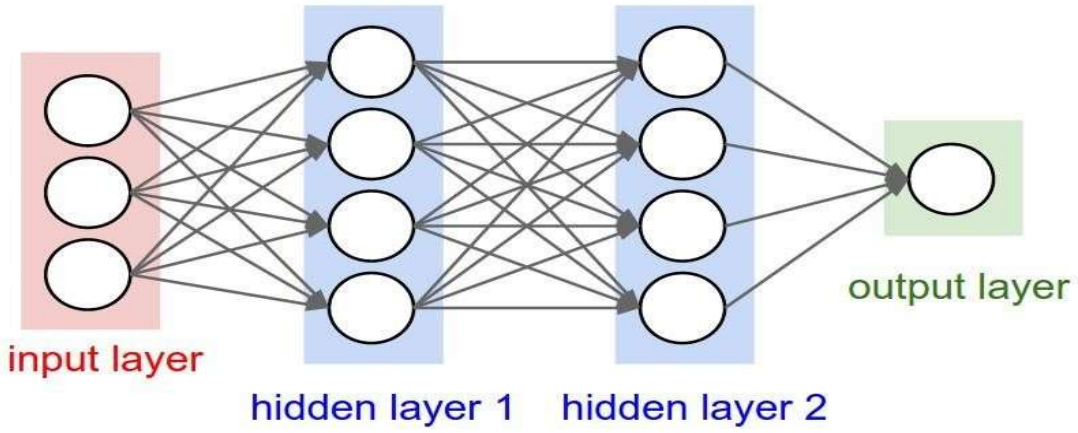
INTRODUCTION

Artificial intelligence, machine learning, and deep learning

- **Artificial intelligence** is a broad term that refers to the ability of machines to perform tasks that are typically associated with human intelligence, such as learning, reasoning, and problem-solving.
- **Machine learning** is a subset of AI that involves the development of algorithms that can learn from data without being explicitly programmed. Machine learning algorithms are trained on large datasets, and they can then be used to make predictions or decisions about new data.
- **Deep learning** is a subset of machine learning that uses artificial neural networks to learn from data. Neural networks are inspired by the human brain, and they can be used to solve complex problems that would be difficult or impossible to solve with traditional machine learning algorithms.

- **Artificial neural networks:** are built on the principles of the structure and operation of human neurons. It is also known as neural networks or neural nets. An artificial neural network's input layer, which is the first layer, receives input from external sources and passes it on to the hidden layer, which is the second layer. Each neuron in the hidden layer gets information from the neurons in the previous layer, computes the weighted total, and then transfers it to the neurons in the next layer.



- **Keras**, Keras is a high-level, deep learning API developed by Google for implementing neural networks. It is written in Python and is used to make the implementation of neural networks easy. It also supports multiple backend neural network computation. Keras is relatively easy to learn and work with because it provides a python frontend with a high level of abstraction while having the option of multiple back-ends for computation purposes. This makes Keras slower than other deep learning frameworks, but extremely beginner-friendly.

Keras allows you to switch between different back ends. The frameworks supported by Keras are:

- Tensorflow

- Theano
- PlaidML
- MXNet
- CNTK (Microsoft Cognitive Toolkit)

- **TensorFlow**: Is an open-source library developed by Google primarily for deep learning applications. It also supports traditional machine learning. Tensor Flow was originally developed for large numerical computations without keeping deep learning in mind.

- **Every project/experiment is done by the following steps:**

1. Import the libraries and load the dataset

First, we are going to import all the modules that we are going to need for training our model. The Keras library already contains some datasets. So we can easily import the dataset and start working with it.

2. Preprocess the data

The image data cannot be fed directly into the model so we need to **perform some operations and process the data** to make it ready for our neural network.

3. Create the model

Now we will **create our CNN model** in Python data science project. A CNN model generally consists of convolutional and pooling layers. It works better for data that are represented as grid structures, this is the reason why CNN works well for image classification problems.

4. Train the model

The **model.fit() function** of Keras will start the training of the model. It **takes the training data, validation data, epochs, and batch size.**It takes some time to train the model. After training, we save the weights and model definition in the 'mnist.h5' file.

5. Evaluate the model

We have 10,000 images in our dataset which will be used to **evaluate how good our model works**. The testing data was not involved in the training of the data therefore, it is new data for our model.

6. Create GUI to predict digits

Now for the GUI, we have created a new file in which we **build an interactive window to draw digits on canvas** and with a button, we can recognize the digit. The Tkinter library comes in the Python standard library. Then we **create the App class** which is responsible for building the GUI for our app. We create a canvas where we can draw by capturing the mouse event and with a button, we trigger the predict_digit() function and display the results.

EXPERIMENT-1

AIM: Implement multilayer perceptron algorithm for MNIST Handwritten Digit Classification.

DESCRIPTION:

One of your assignments is to implement a backpropagation algorithm on a simple MLP architecture. Now, based on that simple BPNN architecture, develop a new model of BPNN capable of tackling a general digit recognition task. Once again, please employ digit datasets taken from sklearn import datasets and split into training (80%) and validation (20%) datasets. Since the size of the sample image is 8 x 8 pixels, so the architecture of your new BPNN model has to be adjusted accordingly. Your python notebook must include the computation of

(A) training accuracy and validation accuracy curves, and

(B) performance analysis of the model based on the associated confusion matrix

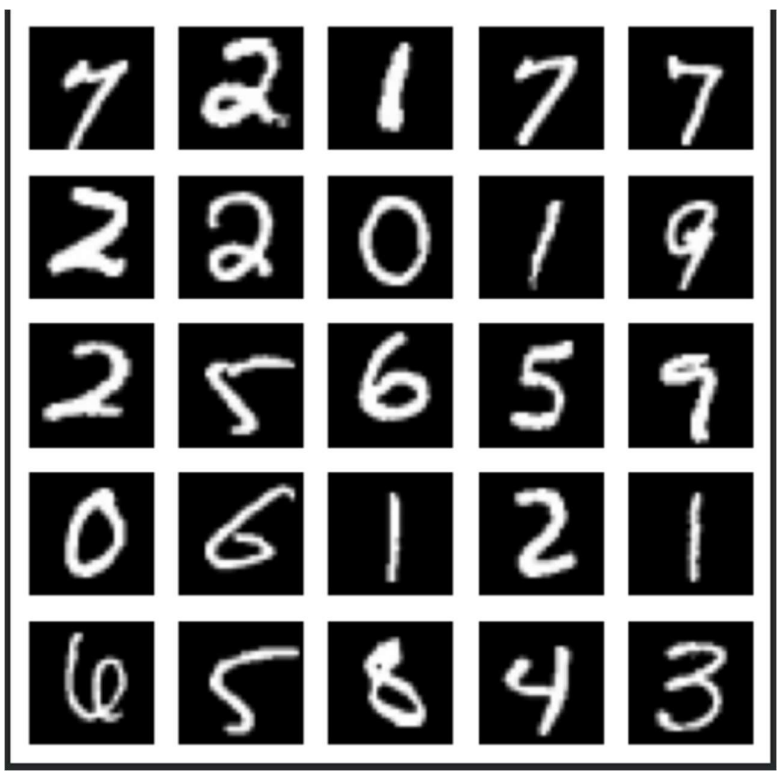
PROGRAM:

```
import numpy as np
from matplotlib import pyplot as plt
from tensorflow.keras.datasets import mnist

# Load MNIST dataset
(x_train, y_train), (x_test, y_test) = mnist.load_data()

# Count training labels
unique, counts = np.unique(y_train, return_counts=True)
print("Train:", dict(zip(unique, counts)))

# Count test labels
unique, counts = np.unique(y_test, return_counts=True)
print("\nTest labels:", dict(zip(unique, counts)))

# Select 25 random images
indexes = np.random.randint(0, x_train.shape[0], size=25)
images = x_train[indexes]
labels = y_train[indexes]

# Plot images
plt.figure(figsize=(5, 5))

for i in range(len(indexes)):
    plt.subplot(5, 5, i + 1)
    image = images[i]
    plt.imshow(image, cmap="gray")
    plt.axis("off")
```

```
# Save and show figure

plt.savefig("mnist_sample.png")
plt.show()

# Close all figures
plt.close("all")
```

OUTPUT:

```
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/mnist.npz
11490434/11490434 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 0s 0us/step
Train: {np.uint8(0): np.int64(5923), np.uint8(1): np.int64(6742), np.uint8(2): np.int64(5958),
 np.uint8(3): np.int64(6131), np.uint8(4): np.int64(5842), np.uint8(5): np.int64(5421), np.uint8(6):
 np.int64(5918), np.uint8(7): np.int64(6265), np.uint8(8): np.int64(5851), np.uint8(9):
 np.int64(5949)}

Test labels: {np.uint8(0): np.int64(980), np.uint8(1): np.int64(1135), np.uint8(2): np.int64(1032),
 np.uint8(3): np.int64(1010), np.uint8(4): np.int64(982), np.uint8(5): np.int64(892), np.uint8(6):
 np.int64(958), np.uint8(7): np.int64(1028), np.uint8(8): np.int64(974), np.uint8(9): np.int64(1009)}
```



EXPERIMENT-2

AIM: Design a neural network for classifying movie reviews (Binary Classification) using IMDB (Internet Movie Data Base) dataset.

DESCRIPTION:

We'll be working with "IMDB dataset", a set of 50,000 highly-polarized reviews from the Internet Movie Database. They are split into 25,000 reviews for training and 25,000 reviews for testing, each set consisting in 50% negative and 50% positive reviews.You should never test a machine learning model on the same data that you used to train it! Just because a model performs well on its training data doesn't mean that it will perform well on data it has never seen, and what you actually care about is your model's performance on new data. Just like the MNIST dataset, the IMDB dataset comes packaged with Keras. It has already been preprocessed: the reviews (sequences of words) have been turned into sequences of integers, where each integer stands for a specific word in a dictionary.

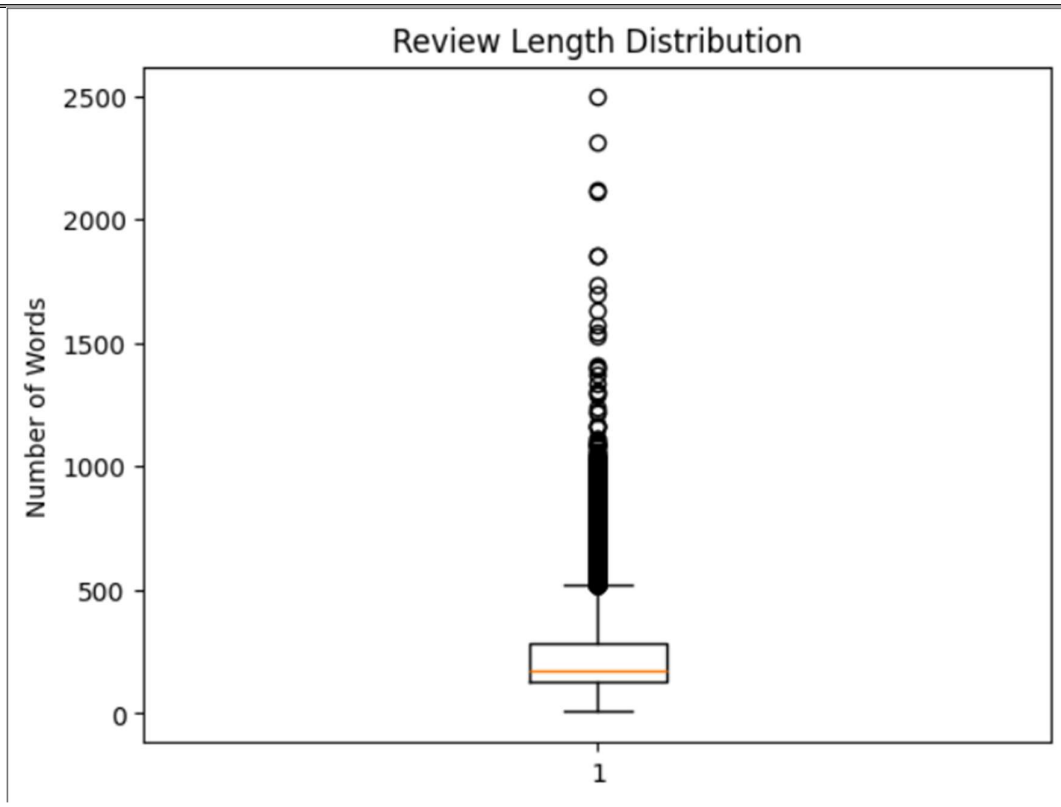
PROGRAM:

```
# Import libraries
import numpy as np
import matplotlib.pyplot as plt
from tensorflow.keras import models
from tensorflow.keras import layers
from tensorflow.keras.datasets import imdb

# Build the model
model = models.Sequential()
model.add(layers.Dense(16, activation='relu', input_shape=(10000,)))
model.add(layers.Dense(16, activation='relu'))
model.add(layers.Dense(1, activation='sigmoid'))

# Load the dataset
(X_train, y_train), (X_test, y_test) = imdb.load_data()

# Combine train and test data
X = np.concatenate((X_train, X_test), axis=0)
y = np.concatenate((y_train, y_test), axis=0)

# Summarize size
print("Training data:")
print(X.shape)
print(y.shape)

# Summarize number of classes
print("Classes:")
print(np.unique(y))
```

```
# Summarize number of words
print("Number of unique words:")
print(len(np.unique(np.hstack(X))))

# Summarize review length
print("Review length:")
result = [len(x) for x in X]
print("Mean %.2f words (Std: %.2f)" % (np.mean(result), np.std(result)))

# Plot review length
plt.boxplot(result)
plt.title("Review Length Distribution")
plt.ylabel("Number of Words")
plt.show()
```

OUTPUT:

```
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/imdb.npz
/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/dense.py:93: UserWarning: Do not pass an `input_shape`/`input_dim` argument to a layer. When using Sequential models, prefer using an `Input(shape)` object as the first layer in the model instead.
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
17464789/17464789 ━━━━━━━━━━━━━━━━━━━━ 1s 0us/step
Training data:
(50000,)
(50000,)
Classes:
[0 1]
Number of unique words:
88585
Review length:
Mean 234.76 words (Std: 172.91)
```


Review Length Distribution
Number of Words
2500
2000
1500
1000
500
0
1

EXPERIMENT-3

AIM: Design a neural Network for classifying news wires (Multi class classification) using Reuters dataset.

DESCRIPTION:

Reuters dataset: We will work with the Reuters dataset, a set of short newswires and their topics, published by Reuters in 1986. It's a simple, widely used toy dataset for text classification. There are 46 different topics; some topics are more represented than others, but each topic has at least 10 examples in the training set. Like IMDB and MNIST, the Reuters dataset comes packaged as part of Keras.

we will build a model to classify Reuters newswires into 46 mutually exclusive topics. Because we have many classes, this problem is an instance of *multiclass classification*; and because each data point should be classified into only one category, the problem is more specifically an instance of single-label, multiclass classification. If each data point could belong to multiple categories (in this case, topics), you'd be facing a multilabel, multiclass classification problem.

PROGRAM:

```
import numpy as np
import matplotlib.pyplot as plt
from tensorflow.keras.datasets import reuters
from tensorflow.keras import models, layers

# Load data
(train_data, train_labels), (test_data, test_labels) = reuters.load_data(num_words=10000)

print(len(train_data))
print(len(test_data))

# Decode example (optional)
word_index = reuters.get_word_index()
reverse_word_index = {value: key for key, value in word_index.items()}

decoded_news = " ".join(
    reverse_word_index.get(i - 3, "?") for i in train_data[10]
)
print(decoded_news)
print(train_labels[10])

# Vectorization function
def vectorize_sequences(sequences, dimension=10000):
    results = np.zeros((len(sequences), dimension))
    for i, sequence in enumerate(sequences):
        results[i, sequence] = 1.0
    return results
```

```
# Vectorize input data
x_train = vectorize_sequences(train_data)
x_test = vectorize_sequences(test_data)

print(x_train.shape)
print(x_train.ndim)

# Example fix: vectorize_sequences expects a list of lists
data = [[1, 2, 4, 9, 0]]
x = vectorize_sequences(data)
print(x.shape, x.ndim)

# One-hot encoding labels
def to_one_hot(labels, dimension=46):
    results = np.zeros((len(labels), dimension))
    for i, label in enumerate(labels):
        results[i, label] = 1.0
    return results

one_hot_train_labels = to_one_hot(train_labels)
one_hot_test_labels = to_one_hot(test_labels)

print(one_hot_train_labels.shape)
print(one_hot_train_labels.ndim)

# Build model
model = models.Sequential()
model.add(layers.Dense(64, activation='relu', input_shape=(10000,)))
model.add(layers.Dense(64, activation='relu'))
model.add(layers.Dense(46, activation='softmax'))

model.compile(
    optimizer='rmsprop',
    loss='categorical_crossentropy',
    metrics=['accuracy']
)

# Validation split
x_val = x_train[:1000]
partial_x_train = x_train[1000:]

y_val = one_hot_train_labels[:1000]
partial_y_train = one_hot_train_labels[1000:]

# Train model
history = model.fit(
    partial_x_train,
    partial_y_train,
    epochs=20,
    batch_size=512,
    validation_data=(x_val, y_val)
)
```

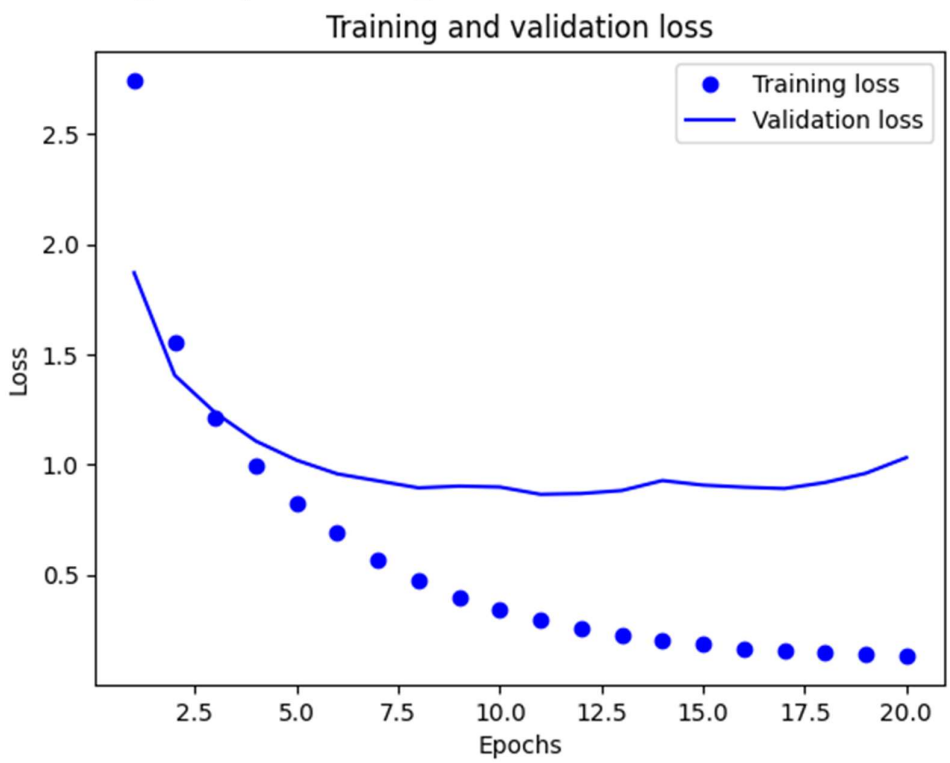
```
# Plot loss
loss = history.history['loss']
val_loss = history.history['val_loss']
epochs = range(1, len(loss) + 1)

plt.plot(epochs, loss, 'bo', label='Training loss')
plt.plot(epochs, val_loss, 'b', label='Validation loss')
plt.title('Training and validation loss')
plt.xlabel('Epochs')
plt.ylabel('Loss')
plt.legend()
plt.show()

# Retrain with optimal epochs
model = models.Sequential()
model.add(layers.Dense(64, activation='relu', input_shape=(10000,)))
model.add(layers.Dense(64, activation='relu'))
model.add(layers.Dense(46, activation='softmax'))

model.compile(
    optimizer='rmsprop',
    loss='categorical_crossentropy',
    metrics=['accuracy']
)

model.fit(
    partial_x_train,
    partial_y_train,
    epochs=8,
    batch_size=512,
    validation_data=(x_val, y_val)
)

# Evaluate
results = model.evaluate(x_test, one_hot_test_labels)

print("Test loss, Test accuracy:", results)
```

OUTPUT:

Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/reuters.npz
2110848/2110848 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ **0s** 0us/step
8982
2246
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/reuters_word_index.json
550378/550378 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ **0s** 0us/step
? period ended december 31 shr profit 11 cts vs loss 24 cts net profit 224 271 vs loss 511 349 revs 7 258 688 vs 7 200 349 reuter 3
3
(8982, 10000)

2
(1, 10000) 2
(8982, 46)
2
/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/dense.py:93: UserWarning: Do not pass an `input_shape`/`input_dim` argument to a layer. When using Sequential models, prefer using an `Input(shape)` object as the first layer in the model instead.
super().__init__(activity_regularizer=activity_regularizer, **kwargs)
Epoch 1/20
16/16 ━━━━━━━━━━━━━━━━━━━━ **2s** 100ms/step - accuracy: 0.3764 - loss: 3.2221 - val_accuracy: 0.6120 - val_loss: 1.8702
Epoch 2/20
16/16 ━━━━━━━━━━━━━━━━━━━━ **2s** 90ms/step - accuracy: 0.6594 - loss: 1.6464 - val_accuracy: 0.6850 - val_loss: 1.4051
Epoch 3/20

Epoch 20/20
16/16 ━━━━━━━━━━━━━━━━━━━━ **1s** 54ms/step - accuracy: 0.9586 - loss: 0.1250 - val_accuracy: 0.7970 - val_loss: 1.0316



Epoch 1/8
16/16 ━━━━━━━━━━━━━━━━━━━━ **2s** 69ms/step - accuracy: 0.3535 - loss: 3.2925 - val_accuracy: 0.6130 - val_loss: 1.9469

Epoch 8/8
16/16 ━━━━━━━━━━━━━━━━━━━━ **1s** 52ms/step - accuracy: 0.9022 - loss: 0.4887 - val_accuracy: 0.8050 - val_loss: 0.8829
71/71 ━━━━━━━━━━━━━━━━━━━━ **0s** 4ms/step - accuracy: 0.7949 - loss: 0.9332
Test loss, Test accuracy: [0.9599744081497192, 0.7858415246009827]

EXPERIMENT-4

AIM: Design a neural network for predicting house prices using Boston Housing Price dataset.

DESCRIPTION:

This project aims to predict the median value of owner-occupied homes in the Boston area using deep learning techniques. The model is trained on the Boston Housing dataset, which consists of various features such as crime rate, average number of rooms, and accessibility to radial highways. This project demonstrates the effectiveness of deep learning in solving regression problems, specifically in the case of predicting housing prices. Further improvements can be made by exploring different neural network architectures and hyperparameter tuning.

PROGRAM:

```
# Step 1: Import required libraries

import numpy as np

import tensorflow as tf

from tensorflow import keras

from tensorflow.keras import layers

from tensorflow.keras.datasets import boston_housing

import matplotlib.pyplot as plt


# Step 2: Load the Boston Housing dataset

(train_data, train_targets), (test_data, test_targets) = boston_housing.load_data()


print("Training data shape:", train_data.shape)

print("Test data shape:", test_data.shape)


# Step 3: Normalize the data (important for Neural Networks)

mean = train_data.mean(axis=0)

std = train_data.std(axis=0)


train_data = (train_data - mean) / std

test_data = (test_data - mean) / std
```

```
# Step 4: Build the Neural Network model
model = keras.Sequential([
    layers.Dense(64, activation='relu', input_shape=(train_data.shape[1],)),
    layers.Dense(64, activation='relu'),
    layers.Dense(1)  # Output layer for regression
])

# Step 5: Compile the model
model.compile(
    optimizer='adam',
    loss='mse',
    metrics=['mae']
)

# Step 6: Train the model
history = model.fit(
    train_data,
    train_targets,
    epochs=100,
    batch_size=16,
    validation_split=0.2,
    verbose=1
)

# Step 7: Evaluate the model
test_loss, test_mae = model.evaluate(test_data, test_targets)
print("\nTest MAE:", test_mae)
```

```
# Step 8: Make predictions
predictions = model.predict(test_data)

print("\nFirst 5 predictions:")
print(predictions[:5])

print("\nActual house prices:")
print(test_targets[:5])

# Step 9: Plot Training and Validation Loss
plt.figure(figsize=(8, 5))
plt.plot(history.history['loss'], label='Training Loss')
plt.plot(history.history['val_loss'], label='Validation Loss')
plt.xlabel('Epochs')
plt.ylabel('Mean Squared Error')
plt.title('Training vs Validation Loss')
plt.legend()
plt.show()
```

OUTPUT:

```
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/boston_housing.npz
57026/57026 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 0s 1us/step
Training data shape: (404, 13)
Test data shape: (102, 13)
/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/dense.py:93: UserWarning: Do not pass an `input_shape`/`input_dim` argument to a layer. When using Sequential models, prefer using an `Input(shape)` object as the first layer in the model instead.
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
Epoch 1/100
21/21 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 3s 32ms/step - loss: 552.2672 - mae: 21.6493 - val_loss: 547.4504 - val_mae: 21.6055
```

Epoch 2/100

21/21 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - loss: 454.0424 - mae: 19.4634 - val_loss: 411.1283 - val_mae: 18.3274

Epoch 3/100

21/21 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - loss: 305.0794 - mae: 15.3025 - val_loss: 227.8982 - val_mae: 12.7506

Epoch 100/100

21/21 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 6.3196 - mae: 1.7443 - val_loss: 15.0501 - val_mae: 2.6755

4/4 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 0s 12ms/step - loss: 16.5313 - mae: 2.7127

Test MAE: 2.920720338821411

4/4 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 0s 20ms/step

First 5 predictions:

[[8.292983]

 [18.221378]

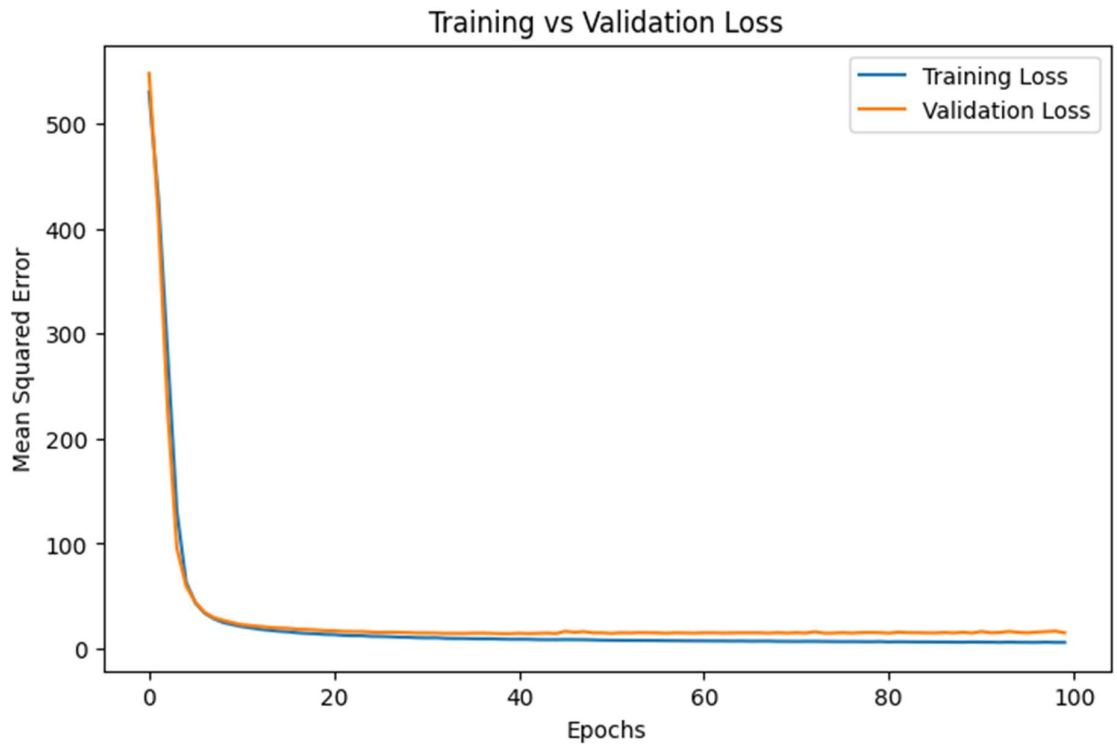
 [21.360199]

 [32.150787]

 [24.964174]]

Actual house prices:

[7.2 18.8 19. 27. 22.2]


Training vs Validation Loss
Training Loss
Validation Loss
Mean Squared Error
500
400
300
200
100
0
0
20
40
60
80
100
Epochs

EXPERIMENT-5

AIM: Build a Convolution Neural Network for MNIST Hand written Digit Classification.

DESCRIPTION:

The MNIST handwritten digit classification problem is a standard dataset used in computer vision and deep learning.Although the dataset is effectively solved, it can be used as the basis for learning and practicing how to develop, evaluate, and use convolutional deep learning neural networks for image classification from scratch. This includes how to develop a robust test harness for estimating the performance of the model, how to explore improvements to the model, and how to save the model and later load it to make predictions on new data.

- How to develop a test harness to develop a robust evaluation of a model and establish a baseline of performance for a classification task.
- How to explore extensions to a baseline model to improve learning and model capacity.
- How to develop a finalized model, evaluate the performance of the final model, and use it to make predictions on new images.

PROGRAM:

```
import tensorflow as tf

from tensorflow.keras.datasets import mnist

from tensorflow.keras.models import Sequential

from tensorflow.keras.layers import Conv2D, MaxPooling2D, Flatten, Dense, Dropout

from tensorflow.keras.utils import to_categorical

import numpy as np

import matplotlib.pyplot as plt


# 1. Load the MNIST data

(train_images, train_labels), (test_images, test_labels) = mnist.load_data()


# 2. Preprocess the Data

# Reshape images to (batch_size, height, width, channels)

train_images = train_images.reshape((60000, 28, 28, 1)).astype('float32') / 255

test_images = test_images.reshape((10000, 28, 28, 1)).astype('float32') / 255
```

```
# Convert labels to one-hot encoding
train_labels = to_categorical(train_labels)
test_labels = to_categorical(test_labels)

# 3. Build the CNN Model
model = Sequential()

# First Conv block
model.add(Conv2D(32, (3, 3), activation='relu', input_shape=(28, 28, 1)))
model.add(MaxPooling2D((2, 2)))

# Second Conv block
model.add(Conv2D(64, (3, 3), activation='relu'))
model.add(MaxPooling2D((2, 2)))

# Flatten layer
model.add(Flatten())

# Fully connected layers
model.add(Dense(128, activation='relu'))
model.add(Dropout(0.5))  # Dropout for regularization
model.add(Dense(10, activation='softmax'))  # Output layer for 10 classes

# 4. Compile the Model
model.compile(
    optimizer='adam',
    loss='categorical_crossentropy',
    metrics=['accuracy']
)
```

```
# Display model summary
model.summary()

# 5. Train the Model
history = model.fit(
    train_images,
    train_labels,
    epochs=5,
    batch_size=64,
    validation_data=(test_images, test_labels)
)

# 6. Evaluate the Model
test_loss, test_acc = model.evaluate(test_images, test_labels, verbose=2)
print(f"\nTest accuracy: {test_acc * 100:.2f}%")

# 7. Visualize Results
plt.figure(figsize=(12, 4))

# Accuracy Plot
plt.subplot(1, 2, 1)
plt.plot(history.history['accuracy'], label='Training Accuracy')
plt.plot(history.history['val_accuracy'], label='Validation Accuracy')
plt.title('Model Accuracy')
plt.xlabel('Epoch')
plt.ylabel('Accuracy')
plt.legend()

# Loss Plot
plt.subplot(1, 2, 2)
```

```
plt.plot(history.history['loss'], label='Training Loss')

plt.plot(history.history['val_loss'], label='Validation Loss')

plt.title('Model Loss')

plt.xlabel('Epoch')

plt.ylabel('Loss')

plt.legend()


plt.tight_layout()

plt.show()
```

OUTPUT:

Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/mnist.npz

11490434/11490434 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 1s 0us/step

/usr/local/lib/python3.12/dist-packages/keras/src/layers/convolutional/base_conv.py:113: UserWarning: Do not pass an `input_shape`/`input_dim` argument to a layer. When using Sequential models, prefer using an `Input(shape)` object as the first layer in the model instead.

super().__init__(activity_regularizer=activity_regularizer, **kwargs)

Model: "sequential"

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 26, 26, 32)	320
max_pooling2d (MaxPooling2D)	(None, 13, 13, 32)	0
conv2d_1 (Conv2D)	(None, 11, 11, 64)	18,496
max_pooling2d_1 (MaxPooling2D)	(None, 5, 5, 64)	0
flatten (Flatten)	(None, 1600)	0
dense (Dense)	(None, 128)	204,928
dropout (Dropout)	(None, 128)	0
dense_1 (Dense)	(None, 10)	1,290

Total params: 225,034 (879.04 KB)

Trainable params: 225,034 (879.04 KB)

Non-trainable params: 0 (0.00 B)

Epoch 1/5

938/938 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 57s 58ms/step - accuracy: 0.8408 - loss: 0.4947 - val_accuracy: 0.9842 - val_loss: 0.0484

Epoch 2/5

938/938 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 52s 55ms/step - accuracy: 0.9743 - loss: 0.0853 - val_accuracy: 0.9879 - val_loss: 0.0347

Epoch 3/5

938/938 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 50s 54ms/step - accuracy: 0.9810 - loss: 0.0609 - val_accuracy: 0.9900 - val_loss: 0.0302

Epoch 4/5

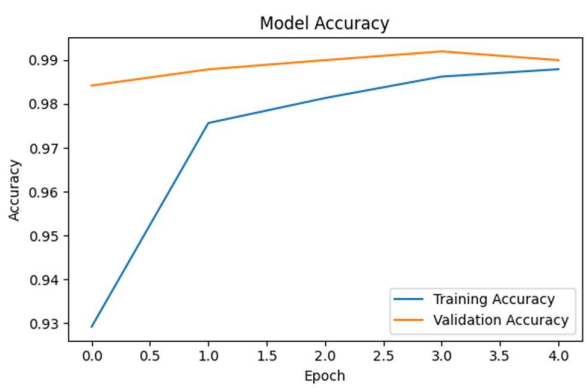
938/938 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 83s 55ms/step - accuracy: 0.9863 - loss: 0.0472 - val_accuracy: 0.9920 - val_loss: 0.0244

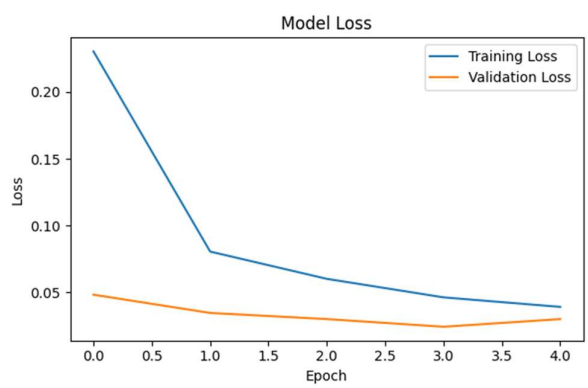
Epoch 5/5

938/938 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 51s 55ms/step - accuracy: 0.9878 - loss: 0.0400 - val_accuracy: 0.9900 - val_loss: 0.0302

313/313 - 3s - 8ms/step - accuracy: 0.9900 - loss: 0.0302

Test accuracy: 99.00%





EXPERIMENT-6

AIM: Build a Convolution Neural Network for simple image (dogs and Cats) Classification.

DESCRIPTION:

The Dogs & Cats is a foundational problem for a basic CNN(convolutional neural network) model which involves classifying images as a dog or a cat.The dataset can be used for learning how to develop,evaluate and use convolutional deep learning neural networks for classification of images. This includes how to develop a robust test harness for estimating the performance of the model, exploring improvements for the model by changing the paramters of the model, saving and loading the model to make predicitions on new data.

PROGRAM:

```
# UPLOAD ZIP

from google.colab import files

uploaded = files.upload()

# IMPORTS

import matplotlib.pyplot as plt

import tensorflow as tf

import pandas as pd

import numpy as np

import warnings

warnings.filterwarnings('ignore')


from tensorflow import keras

from tensorflow.keras.models import Sequential

from tensorflow.keras.layers import Activation, Dropout, Flatten, Dense, Conv2D, MaxPooling2D

from tensorflow.keras.preprocessing.image import ImageDataGenerator, load_img

import os

import matplotlib.image as mpimg

from zipfile import ZipFile


#  EXTRACT ZIP

data_path = 'dogs-vs-cats (6).zip'
```

```
with ZipFile(data_path, 'r') as zip_ref:
    zip_ref.extractall()
print('The data set has been extracted.')
# FIND DATASET FOLDER
dataset_path = None
for root, dirs, files in os.walk(os.getcwd()):
    if 'cats' in dirs and 'dogs' in dirs:
        dataset_path = root
        break
print("Dataset path:", dataset_path)
path = dataset_path

#DISPLAY SAMPLE IMAGES
fig = plt.gcf()
fig.set_size_inches(16, 16)
cat_dir = os.path.join(path, 'cats')
dog_dir = os.path.join(path, 'dogs')
cat_names = os.listdir(cat_dir)
dog_names = os.listdir(dog_dir)
pic_index = 210
cat_images = [
    os.path.join(cat_dir, fname)
    for fname in cat_names[pic_index - 8:pic_index]
]
dog_images = [
    os.path.join(dog_dir, fname)
    for fname in dog_names[pic_index - 8:pic_index]
]
```

```
for i, img_path in enumerate(cat_images + dog_images):
    sp = plt.subplot(4, 4, i + 1)
    sp.axis('off')
    img = mpimg.imread(img_path)
    plt.imshow(img)
plt.show()

#IMAGE PARAMETERS
IMG_HEIGHT = 150
IMG_WIDTH = 150
BATCH_SIZE = 32
#DATA GENERATOR
train_datagen = ImageDataGenerator(
    rescale=1.0 / 255,
    validation_split=0.2,
    rotation_range=20,
    zoom_range=0.2,
    horizontal_flip=True
)
train_generator = train_datagen.flow_from_directory(
    path,
    target_size=(IMG_HEIGHT, IMG_WIDTH),
    batch_size=BATCH_SIZE,
    class_mode='binary',
    subset='training'
)
validation_generator = train_datagen.flow_from_directory(
    path,
    target_size=(IMG_HEIGHT, IMG_WIDTH),
    batch_size=BATCH_SIZE,
```

```
    class_mode='binary',
    subset='validation'
)

#BUILD CNN MODEL
model = Sequential([
    Conv2D(32, (3, 3), activation='relu', input_shape=(IMG_HEIGHT, IMG_WIDTH, 3)),
    MaxPooling2D(2, 2),
    Conv2D(64, (3, 3), activation='relu'),
    MaxPooling2D(2, 2),
    Conv2D(128, (3, 3), activation='relu'),
    MaxPooling2D(2, 2),
    Flatten(),
    Dense(128, activation='relu'),
    Dropout(0.5),
    Dense(1, activation='sigmoid')
])
# COMPILE MODEL
model.compile(
    optimizer='adam',
    loss='binary_crossentropy',
    metrics=['accuracy']
)
model.summary()
#TRAIN MODEL
history = model.fit(
    train_generator,
    epochs=10,
    validation_data=validation_generator
)
```

```
#PLOT ACCURACY & LOSS
acc = history.history['accuracy']
val_acc = history.history['val_accuracy']
loss = history.history['loss']
val_loss = history.history['val_loss']
epochs_range = range(len(acc))
plt.figure(figsize=(12, 5))
plt.subplot(1, 2, 1)
plt.plot(epochs_range, acc, label='Training Accuracy')
plt.plot(epochs_range, val_acc, label='Validation Accuracy')
plt.legend()
plt.title('Accuracy')
plt.subplot(1, 2, 2)
plt.plot(epochs_range, loss, label='Training Loss')
plt.plot(epochs_range, val_loss, label='Validation Loss')
plt.legend()
plt.title('Loss')
plt.show()
#  PREDICTION (FROM DATASET IMAGE)
from tensorflow.keras.preprocessing import image
img_path = cat_images[0]
img = image.load_img(img_path, target_size=(IMG_HEIGHT, IMG_WIDTH))
img_array = image.img_to_array(img)
img_array = np.expand_dims(img_array, axis=0)
img_array /= 255.0
prediction = model.predict(img_array)
if prediction[0][0] > 0.5:
    print("Prediction: Dog 🐶")
```

```
else:
    print("Prediction: Cat 😺")
#  PREDICTION FROM UPLOADED IMAGE
uploaded = files.upload()

for fname in uploaded.keys():
    os.rename(fname, "input_image.jpg")
    break
img_path = "input_image.jpg"
img = image.load_img(img_path, target_size=(IMG_HEIGHT, IMG_WIDTH))
img_array = image.img_to_array(img)
img_array = np.expand_dims(img_array, axis=0)
img_array /= 255.0
prediction = model.predict(img_array)
print("Prediction value:", prediction[0][0])
if prediction[0][0] > 0.5:
    print("Prediction: Dog 🐶")
else:
    print("Prediction: Cat 😺")
```

OUTPUT:

Upload widget is only available when the cell has been executed in the current browser session. Please rerun this cell to enable.

Saving dogs-vs-cats.zip to dogs-vs-cats.zip

The data set has been extracted.

```
Dataset path: /content/dogs-vs-cats
```

```
Found 400 images belonging to 2 classes.
Found 100 images belonging to 2 classes.
Model: "sequential_2"
```

Layer (type)	Output Shape	Param #
conv2d_6 (Conv2D)	(None, 148, 148, 32)	896
max_pooling2d_6 (MaxPooling2D)	(None, 74, 74, 32)	0
conv2d_7 (Conv2D)	(None, 72, 72, 64)	18,496
max_pooling2d_7 (MaxPooling2D)	(None, 36, 36, 64)	0
conv2d_8 (Conv2D)	(None, 34, 34, 128)	73,856
max_pooling2d_8 (MaxPooling2D)	(None, 17, 17, 128)	0
flatten_2 (Flatten)	(None, 36992)	0
dense_4 (Dense)	(None, 128)	4,735,104
dropout_2 (Dropout)	(None, 128)	0
dense_5 (Dense)	(None, 1)	129

```
Total params: 4,828,481 (18.42 MB)
Trainable params: 4,828,481 (18.42 MB)
Non-trainable params: 0 (0.00 B)
```

Total params: 4,828,481 (18.42 MB)

Trainable params: 4,828,481 (18.42 MB)

Non-trainable params: 0 (0.00 B)

Epoch 1/10

13/13 ━━━━━━━━━━━━━━━━━━━━ 24s 2s/step - accuracy: 0.4643 - loss: 0.9574 - val_accuracy: 0.5000 - val_loss: 0.6937

Epoch 2/10

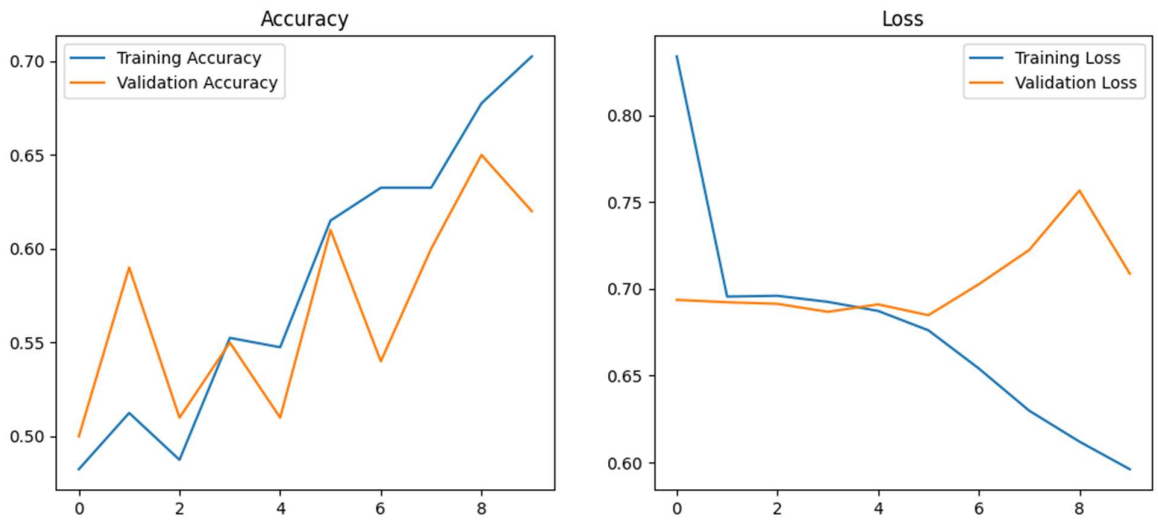
13/13 ━━━━━━━━━━━━━━━━━━━━ 20s 2s/step - accuracy: 0.5261 - loss: 0.6946 - val_accuracy: 0.5900 - val_loss: 0.6923

Epoch 3/10

13/13 ━━━━━━━━━━━━━━━━━━━━ 21s 2s/step - accuracy: 0.5350 - loss: 0.6927 - val_accuracy: 0.5100 - val_loss: 0.6914

Epoch 10/10

13/13 ━━━━━━━━━━━━━━━━━━━━ 20s 2s/step - accuracy: 0.6972 - loss: 0.6132 - val_accuracy: 0.6200 - val_loss: 0.7088



1/1 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 0s 114ms/step

Prediction: Dog 🐶

Upload widget is only available when the cell has been executed in the current browser session. Please rerun this cell to enable.

Saving 45.jpg to 45.jpg

1/1 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 0s 50ms/step

Prediction value: 0.86397237

Prediction: Dog 🐶

EXPERIMENT-7

AIM: Use a pre-trained convolution neural network (VGG16) for image classification.

DESCRIPTION:

To use a pre-trained VGG16 convolutional neural network for image classification, you would: load the pre-trained VGG16 model, preprocess your new images to match the format expected by the model, then pass the images through the model to obtain predictions about their class labels; essentially leveraging the features learned from a large dataset (like ImageNet) to classify images from your specific domain by replacing the final fully connected layers with ones tailored to your task, a process called transfer learning.

PROGRAM:

```
# IMPORTS
import tensorflow as tf
from tensorflow.keras.applications import VGG16
from tensorflow.keras.models import Model
from tensorflow.keras.layers import Flatten, Dense, Dropout
from tensorflow.keras.utils import plot_model
from tensorflow.keras.preprocessing.image import load_img
from google.colab import files
import matplotlib.pyplot as plt
#LOAD PRETRAINED VGG16
base_model = VGG16(
    weights='imagenet',
    include_top=False,
    input_shape=(224, 224, 3)
)
# FREEZE BASE MODEL LAYERS
for layer in base_model.layers:
    layer.trainable = False
# ADD CUSTOM CLASSIFICATION LAYERS
x = base_model.output
x = Flatten()(x)
```

```
x = Dense(256, activation='relu')(x)
x = Dropout(0.5)(x)
output = Dense(3, activation='softmax')(x)
# CREATE FINAL MODEL
model = Model(
    inputs=base_model.input,
    outputs=output
)
# COMPILE MODEL
model.compile(
    optimizer='adam',
    loss='categorical_crossentropy',
    metrics=['accuracy']
)
# MODEL SUMMARY
model.summary()
#  VISUALIZE MODEL ARCHITECTURE
plot_model(
    model,
    show_shapes=True,
    show_layer_names=True
)
#UPLOAD IMAGE
uploaded = files.upload()
# LOAD AND DISPLAY IMAGE
img = load_img('dog1.jpg', target_size=(224, 224))
plt.imshow(img)
plt.axis('off')
plt.show()
```

OUTPUT:

Downloading data from https://storage.googleapis.com/tensorflow/keras-applications/vgg16/vgg16_weights_tf_dim_ordering_tf_kernels_notop.h5

58889256/58889256 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 0s 0us/step

Model: "functional"

Layer (type)	Output Shape	Param #
input_layer (InputLayer)	(None, 224, 224, 3)	0
block1_conv1 (Conv2D)	(None, 224, 224, 64)	1,792
block1_conv2 (Conv2D)	(None, 224, 224, 64)	36,928
block1_pool (MaxPooling2D)	(None, 112, 112, 64)	0
block2_conv1 (Conv2D)	(None, 112, 112, 128)	73,856
block2_conv2 (Conv2D)	(None, 112, 112, 128)	147,584
block2_pool (MaxPooling2D)	(None, 56, 56, 128)	0
block3_conv1 (Conv2D)	(None, 56, 56, 256)	295,168
block3_conv2 (Conv2D)	(None, 56, 56, 256)	590,080
block3_conv3 (Conv2D)	(None, 56, 56, 256)	590,080
block3_pool (MaxPooling2D)	(None, 28, 28, 256)	0
block4_conv1 (Conv2D)	(None, 28, 28, 512)	1,180,160
block4_conv2 (Conv2D)	(None, 28, 28, 512)	2,359,808
block4_conv3 (Conv2D)	(None, 28, 28, 512)	2,359,808
block4_pool (MaxPooling2D)	(None, 14, 14, 512)	0

block5_conv1 (Conv2D)	(None, 14, 14, 512)	2,359,808
block5_conv2 (Conv2D)	(None, 14, 14, 512)	2,359,808
block5_conv3 (Conv2D)	(None, 14, 14, 512)	2,359,808
block5_pool (MaxPooling2D)	(None, 7, 7, 512)	0
flatten (Flatten)	(None, 25088)	0
dense (Dense)	(None, 256)	6,422,784
dropout (Dropout)	(None, 256)	0
dense_1 (Dense)	(None, 3)	771

Total params: 21,138,243 (80.64 MB)
Trainable params: 6,423,555 (24.50 MB)
Non-trainable params: 14,714,688 (56.13 MB)

Choose Files dog1.jpg

dog1.jpg(image/jpeg) - 10350 bytes, last modified: 2/28/2026 - 100% done
Saving dog1.jpg to dog1.jpg

EXPERIMENT-8

AIM: : Implement one hot encoding of words or characters.

DESCRIPTION:
One-hot encoding is the most common, most basic way to turn a token into a vector. You already saw it in action in our initial IMDB and Reuters examples from chapter 3 (done with words, in our case). It consists in associating a unique integer index to every word, then turning this integer index i into a binary vector of size N, the size of the vocabulary, that would be all-zeros except for the i-th entry, which would be 1.

Of course, one-hot encoding can be done at the character level as well. To unambiguously drive home what one-hot encoding is and how to implement it, here are two toy examples of one-hot encoding: one for words, the other for characters.

PROGRAM:

```
import numpy as np


# Sample sentences
samples = {
    'Jupiter has 79 known moons .',
    'Neptune has 14 confirmed moons !'
}


# Step 1: Create token index (vocabulary)
token_index = {}
counter = 0


for sample in samples:
    for considered_word in sample.split():
        if considered_word not in token_index:
            counter += 1
            token_index[considered_word] = counter


print("Token Index:")
```

```
print(token_index)

# Step 2: Set maximum sentence length
max_length = 6

# Step 3: Create zero tensor (samples × max_length × vocabulary_size+1)
results = np.zeros(
    (len(samples), max_length, max(token_index.values()) + 1)
)

# Step 4: One-hot encoding
for i, sample in enumerate(samples):
    for j, considered_word in enumerate(sample.split()):
        if j < max_length:
            index = token_index[considered_word]
            results[i, j, index] = 1

# Step 5: Display results
print("\nOne-Hot Encoded Tensor:")
print(results)
```

OUTPUT:

```
Token Index:
{'Neptune': 1, 'has': 2, '14': 3, 'confirmed': 4, 'moons': 5, '!': 6, 'Jupiter': 7, '79': 8, 'known': 9, '.': 10}
One-Hot Encoded Tensor:
[[[0. 1. 0. 0. 0. 0. 0. 0. 0. 0. 0.]
  [0. 0. 1. 0. 0. 0. 0. 0. 0. 0. 0.]
  [0. 0. 0. 1. 0. 0. 0. 0. 0. 0. 0.]
  [0. 0. 0. 0. 1. 0. 0. 0. 0. 0. 0.]
  [0. 0. 0. 0. 0. 1. 0. 0. 0. 0. 0.]
  [0. 0. 0. 0. 0. 0. 1. 0. 0. 0. 0.]]
```

```
[[0. 0. 0. 0. 0. 0. 0. 1. 0. 0. 0.]
 [0. 0. 1. 0. 0. 0. 0. 0. 0. 0. 0.]
 [0. 0. 0. 0. 0. 0. 0. 0. 1. 0. 0.]
 [0. 0. 0. 0. 0. 0. 0. 0. 0. 1. 0.]
 [0. 0. 0. 0. 0. 1. 0. 0. 0. 0. 0.]
 [0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 1.]]]
```

EXPERIMENT-9

AIM: Implement word embeddings for IMDB dataset.

DESCRIPTION:
Word embedding is essential in natural language processing with deep learning. This technique allows the network to learn about the meaning of the words. In this post, we classify movie reviews in the IMDB dataset as positive or negative, and provide a visual illustration of embedding.

Today is to train a neural network to find out whether some text is globally positive or negative, a task called sentiment analysis.

The first layer of our neural network will perform an operation called word embedding, which is essential in NLP with deep learning.

The IMDB database: We will work with the IMDB dataset, which contains 25,000 movie reviews from IMDB. Each review is labeled as positive or negative from the rating provided by users together with their reviews.

PROGRAM:

```
# Set random seeds for reproducibility

from numpy.random import seed

seed(0xdeadbeef)


import tensorflow as tf

tf.random.set_seed(0xdeadbeef)


from tensorflow import keras

import matplotlib.pyplot as plt


# Load IMDB dataset

imdb = keras.datasets.imdb

num_words = 20000


(train_data, train_labels), (test_data, test_labels) = imdb.load_data(

    seed=1,
```

```
    num_words=num_words
)
print(train_data[0])
print("Label:", train_labels[0])

# Get word index dictionary
vocabulary = imdb.get_word_index()

# Shift indices by 3 (reserve special tokens)
vocabulary = {k: (v + 3) for k, v in vocabulary.items()}
vocabulary["<PAD>"] = 0
vocabulary["<START>"] = 1
vocabulary["<UNK>"] = 2
vocabulary["<UNUSED>"] = 3

# Create reverse dictionary (number → word)
index = dict((value, key) for (key, value) in vocabulary.items())

# Function to convert numbers back to words
def decode_review(text):
    return ' '.join([index.get(i, '?') for i in text])

# Print decoded review
print(decode_review(train_data[0]))

# Pad sequences to same length (256 words)
train_data = keras.preprocessing.sequence.pad_sequences(
    train_data,
    value=vocabulary["<PAD>"],
    padding='post',
```

```
    maxlen=256
)

test_data = keras.preprocessing.sequence.pad_sequences(
    test_data,
    value=vocabulary["<PAD>"],
    padding='post',
    maxlen=256
)
# Build the model
model = keras.Sequential()

# Embedding layer (word → vector)
model.add(keras.layers.Embedding(len(vocabulary), 2, input_length=256))

# Flatten output
model.add(keras.layers.Flatten())

# Add dropout to prevent overfitting
model.add(keras.layers.Dropout(0.5))

# Hidden dense layer
model.add(keras.layers.Dense(5))

# Output layer (binary classification)
model.add(keras.layers.Dense(1, activation='sigmoid'))
model.summary()

# Compile model
model.compile(
```

```
    optimizer='adam',
    loss='binary_crossentropy',
    metrics=['accuracy']
)

# Train model
history = model.fit(
    train_data,
    train_labels,
    epochs=5,
    batch_size=100,
    validation_data=(test_data, test_labels),
    verbose=1
)

# Plot accuracy graph
def plot_accuracy(history):
    acc = history.history['accuracy']
    val_acc = history.history['val_accuracy']
    epochs = range(len(acc))
    plt.plot(epochs, acc, label='Training Accuracy')
    plt.plot(epochs, val_acc, label='Validation Accuracy')
    plt.title('Accuracy')
    plt.xlabel('Epoch')
    plt.legend()
    plt.show()

plot_accuracy(history)
```

OUTPUT:

Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/imdb.npz

17464789/17464789 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 0s 0us/step

Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/imdb_word_index.json

1641221/1641221 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 0s 0us/step

warnings.warn(

Model: "sequential"

Layer (type)	Output Shape	Param #
embedding (Embedding)	?	0 (unbuilt)
flatten (Flatten)	?	0 (unbuilt)
dropout (Dropout)	?	0
dense (Dense)	?	0 (unbuilt)
dense_1 (Dense)	?	0 (unbuilt)

Total params: 0 (0.00 B)

Trainable params: 0 (0.00 B)

Non-trainable params: 0 (0.00 B)

Epoch 1/5

250/250 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 3s 8ms/step - accuracy: 0.5229 - loss: 0.6914 - val_accuracy: 0.7464 - val_loss: 0.6104

Epoch 2/5

250/250 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 2s 8ms/step - accuracy: 0.7699 - loss: 0.5213 - val_accuracy: 0.8608 - val_loss: 0.3541

Epoch 3/5

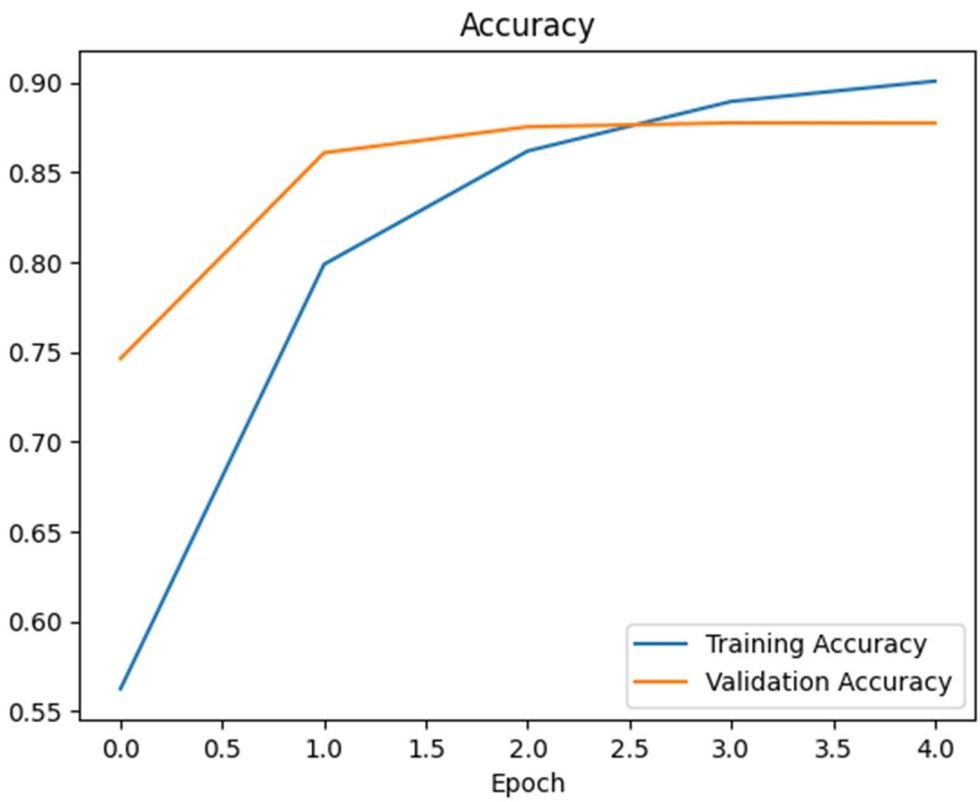
250/250 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 2s 8ms/step - accuracy: 0.8528 - loss: 0.3383 - val_accuracy: 0.8753 - val_loss: 0.3088

Epoch 4/5

250/250 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 3s 10ms/step - accuracy: 0.8867 - loss: 0.2767 - val_accuracy: 0.8774 - val_loss: 0.2966

Epoch 5/5

250/250 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 4s 8ms/step - accuracy: 0.9008 - loss: 0.2395 - val_accuracy: 0.8773 - val_loss: 0.2965


Accuracy
0.90
0.85
0.80
0.75
0.70
0.65
0.60
0.55
0.0
0.5
1.0
1.5
2.0
2.5
3.0
3.5
4.0
Epoch
Training Accuracy
Validation Accuracy

EXPERIMENT-10

AIM: Implement a Recurrent Neural Network for IMDB movie review classification problem.

DESCRIPTION:

The process of building a Sentiment Analysis model using Recurrent Neural Networks (RNNs) to classify movie reviews as positive or negative. Sentiment analysis is a popular Natural Language Processing (NLP) task that involves determining the sentiment or emotion expressed in a piece of text. We will use Python, TensorFlow, and Keras to implement the RNN model and analyze the results.

Recurrent Neural Networks (RNN) are to the rescue when the sequence of information is needed to be captured (another use case may include Time Series, next word prediction, etc.). Due to its internal memory factor, it remembers past sequences along with current input which makes it capable to capture context rather than just individual words.

The IMDB dataset consists of movie reviews from the IMDb website, along with labels indicating whether each review is "positive" or "negative" based on the reviewer's opinion. It is designed for binary sentiment classification, with 25,000 reviews for training and an additional 25,000 for testing. Both the training and testing sets have an equal number of positive and negative reviews, making them balanced for sentiment analysis tasks.

PROGRAM:

```
import numpy as np

import pandas as pd

import tensorflow as tf

import re

import matplotlib.pyplot as plt

import seaborn as sns

import pickle


from tensorflow.keras.layers import SimpleRNN, Dense, Embedding

from tensorflow.keras.datasets import imdb

from tensorflow.keras.models import Sequential

from tensorflow.keras.preprocessing import sequence


# Load IMDB dataset (top 5000 frequent words only)
```

```
vocab_size = 5000
(x_train, y_train), (x_test, y_test) = imdb.load_data(num_words=vocab_size)

print(x_train[0])  # Print first encoded review

# Get word index dictionary
word_idx = imdb.get_word_index()

# Reverse dictionary (number → word)
word_idx = {i: word for word, i in word_idx.items()}

# Print decoded review
print([word_idx.get(i, '?') for i in x_train[0]])

# Print max and min review length
print("Max length of a review:: ",
    len(max((x_train.tolist() + x_test.tolist()), key=len)))

print("Min length of a review:: ",
    len(min((x_train.tolist() + x_test.tolist()), key=len)))

# Pad all reviews to 400 words
max_words = 400
x_train = sequence.pad_sequences(x_train, maxlen=max_words)
x_test = sequence.pad_sequences(x_test, maxlen=max_words)

# Create validation split
x_valid, y_valid = x_train[:64], y_train[:64]
x_train_, y_train_ = x_train[64:], y_train[64:]
```

```
# Set embedding size
embd_len = 32

# Build Simple RNN model
RNN_model = Sequential(name="Simple_RNN")

# Embedding layer (word → vector)
RNN_model.add(Embedding(vocab_size, embd_len, input_length=max_words))

# RNN layer
RNN_model.add(SimpleRNN(128, activation='tanh'))

# Output layer (binary classification)
RNN_model.add(Dense(1, activation='sigmoid'))

# Show model summary
print(RNN_model.summary())

# Compile model
RNN_model.compile(
    loss="binary_crossentropy",
    optimizer='adam',
    metrics=['accuracy']
)

# Train model
history = RNN_model.fit(
    x_train_,
    y_train_,
    batch_size=64,
```

```
    epochs=5,
    verbose=1,
    validation_data=(x_valid, y_valid)
)

# Evaluate on test data
print()
print("Simple_RNN Score---> ",
      RNN_model.evaluate(x_test, y_test, verbose=0))

# Plot accuracy graph
plt.plot(history.history['accuracy'])
plt.plot(history.history['val_accuracy'])
plt.title('Model Accuracy')
plt.ylabel('Accuracy')
plt.xlabel('Epoch')
plt.legend(['Train', 'Validation'], loc='upper left')
plt.show()
```

OUTPUT:

```
KDownloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/imdb.npz
17464789/17464789 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 0s 0us/step
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/imdb_word_index.json
1641221/1641221 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 0s 0us/step
Max length of a review::  2494
Min length of a review::  7
Model: "Simple_RNN"
```

Layer (type)	Output Shape	Param #
embedding (Embedding)	?	0 (unbuilt)
simple_rnn (SimpleRNN)	?	0 (unbuilt)
dense (Dense)	?	0 (unbuilt)

Total params: 0 (0.00 B)
Trainable params: 0 (0.00 B)
Non-trainable params: 0 (0.00 B)

None

Epoch 1/5

390/390 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 71s 177ms/step - accuracy: 0.5522 - loss: 0.6776 - val_accuracy: 0.6094 - val_loss: 0.6216

Epoch 2/5

390/390 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 68s 176ms/step - accuracy: 0.7390 - loss: 0.5211 - val_accuracy: 0.8438 - val_loss: 0.4199

Epoch 3/5

390/390 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 68s 173ms/step - accuracy: 0.8286 - loss: 0.3975 - val_accuracy: 0.7500 - val_loss: 0.5283

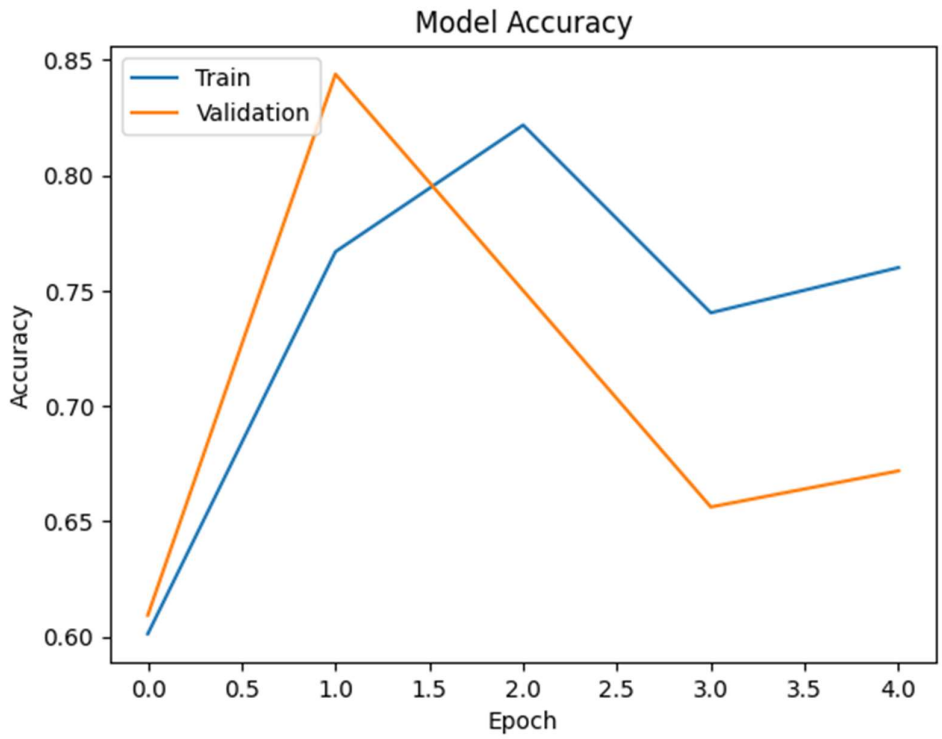
Epoch 4/5

390/390 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 84s 179ms/step - accuracy: 0.7831 - loss: 0.4585 - val_accuracy: 0.6562 - val_loss: 0.5939

Epoch 5/5

390/390 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 80s 174ms/step - accuracy: 0.7615 - loss: 0.4889 - val_accuracy: 0.6719 - val_loss: 0.6045

Simple_RNN Score---> [0.5558714270591736, 0.7087600231170654]


Model Accuracy
Train
Validation
Accuracy
Epoch
0.85
0.80
0.75
0.70
0.65
0.60
0.0
0.5
1.0
1.5
2.0
2.5
3.0
3.5
4.0